

R Basics and Assignment Workflow Guide

Big-Ahh-R-Repo

May 18, 2026

Contents

1	What is R?	2
2	Getting Started	2
2.1	Install R and RStudio	2
2.2	RStudio Layout	2
3	Core R Basics	2
3.1	Objects and Assignment	2
3.2	Data Types	2
3.3	Vectors	2
3.4	Matrices and Lists	3
3.5	Data Frames	3
4	Control Flow and Functions	3
4.1	Conditionals	3
4.2	Loops	3
4.3	Functions	3
5	Useful Data Work	3
5.1	Read and Write Files	3
5.2	Common Base R Operations	4
6	Plotting Basics	4
7	Packages and Reproducibility	4
7.1	Install and Load Packages	4
7.2	Good Habits	4
8	R Markdown for School Assignments	4
8.1	Why R Markdown?	4
8.2	Minimal Structure	4
8.3	Basic R Markdown Example	5
8.4	Render from Console	5
9	Recommended Assignment Workflow	5
10	Debugging and Help	5

11 Quick Reference Table	6
12 Next Steps	6

1 What is R?

R is a language for statistics, data analysis, and visualization. You can use it for homework, projects, and reports.

2 Getting Started

2.1 Install R and RStudio

1. Install R from <https://cran.r-project.org/>.
2. Install RStudio Desktop from <https://posit.co/download/rstudio-desktop/>.

2.2 RStudio Layout

You will mostly use:

- **Script editor:** write .R and .Rmd files.
- **Console:** run code interactively.
- **Environment:** inspect created objects.
- **Plots/Files/Packages/Help:** visualize data and find help.

3 Core R Basics

3.1 Objects and Assignment

Use `<-` for assignment.

```
x <- 10
y <- 5
z <- x + y
```

3.2 Data Types

- Numeric: 3.14
- Integer: 2L
- Character: "hello"
- Logical: TRUE, FALSE
- Factor: categorical values

3.3 Vectors

```
scores <- c(88, 92, 79, 95)
mean(scores)
length(scores)
summary(scores)
```

3.4 Matrices and Lists

```
m <- matrix(1:6, nrow = 2, byrow = TRUE)
student <- list(name = "Alex", id = 101, grades = scores)
```

3.5 Data Frames

```
df <- data.frame(
  name = c("Ana", "Ben", "Cara"),
  exam1 = c(90, 82, 87),
  exam2 = c(93, 79, 91)
)

str(df)
head(df)
df$average <- (df$exam1 + df$exam2) / 2
```

4 Control Flow and Functions

4.1 Conditionals

```
score <- 84
if (score >= 90) {
  grade <- "A"
} else if (score >= 80) {
  grade <- "B"
} else {
  grade <- "C or below"
}
```

4.2 Loops

```
for (i in 1:3) {
  print(i)
}
```

4.3 Functions

```
calc_percent <- function(points, total) {
  (points / total) * 100
}
```

```
calc_percent(45, 50)
```

5 Useful Data Work

5.1 Read and Write Files

```
# Base R
```

```
students <- read.csv("data/raw/students.csv")
write.csv(students, "data/processed/students_clean.csv", row.names = FALSE)
```

5.2 Common Base R Operations

```
subset(df, exam1 >= 85)
transform(df, passed = average >= 70)
aggregate(average ~ name, data = df, FUN = mean)
```

6 Plotting Basics

```
plot(df$exam1, df$exam2,
     main = "Exam 1 vs Exam 2",
     xlab = "Exam 1",
     ylab = "Exam 2")

hist(scores, main = "Score Distribution", xlab = "Score")
boxplot(scores, main = "Scores")
```

7 Packages and Reproducibility

7.1 Install and Load Packages

```
install.packages("rmarkdown")
install.packages("tidyverse")
library(rmarkdown)
library(tidyverse)
```

7.2 Good Habits

- Keep code in scripts, not only in the console.
- Use relative file paths inside the project folder.
- Save outputs (tables/plots) in dedicated folders.
- Comment your logic and cite data sources.

8 R Markdown for School Assignments

8.1 Why R Markdown?

R Markdown combines code, narrative, and output into one reproducible report.

8.2 Minimal Structure

An `.Rmd` file contains:

- YAML header (title/author/date/output)

- Narrative text in Markdown
- Code chunks in R

8.3 Basic R Markdown Example

```
---
title: "Homework 1"
author: "Your Name"
output: html_document
---
```

```
```{r}
x <- c(2, 4, 6, 8)
mean(x)
```
```

8.4 Render from Console

```
rmarkdown::render("report.Rmd")
```

9 Recommended Assignment Workflow

1. Create one project folder per assignment.
2. Put raw data in `data/raw/` and cleaned data in `data/processed/`.
3. Write analysis helpers in `analysis.R`.
4. Write submission in `report.Rmd`.
5. Knit or render before submitting.

10 Debugging and Help

- Read error messages from top to bottom.
- Use `str()`, `head()`, and `summary()` often.
- Use `?function_name` for help pages.
- Search examples in package vignettes and documentation.

11 Quick Reference Table

| Task | Example |
|---------------|--|
| Create vector | <code>c(1, 2, 3)</code> |
| Filter rows | <code>subset(df, score > 80)</code> |
| Add column | <code>df\$new <- ...</code> |
| Read CSV | <code>read.csv("file.csv")</code> |
| Write CSV | <code>write.csv(df, "out.csv", row.names = FALSE)</code> |
| Render report | <code>rmarkdown::render("report.Rmd")</code> |

12 Next Steps

Practice by editing the assignment template in this repository. Start small, run your code frequently, and keep your work reproducible.